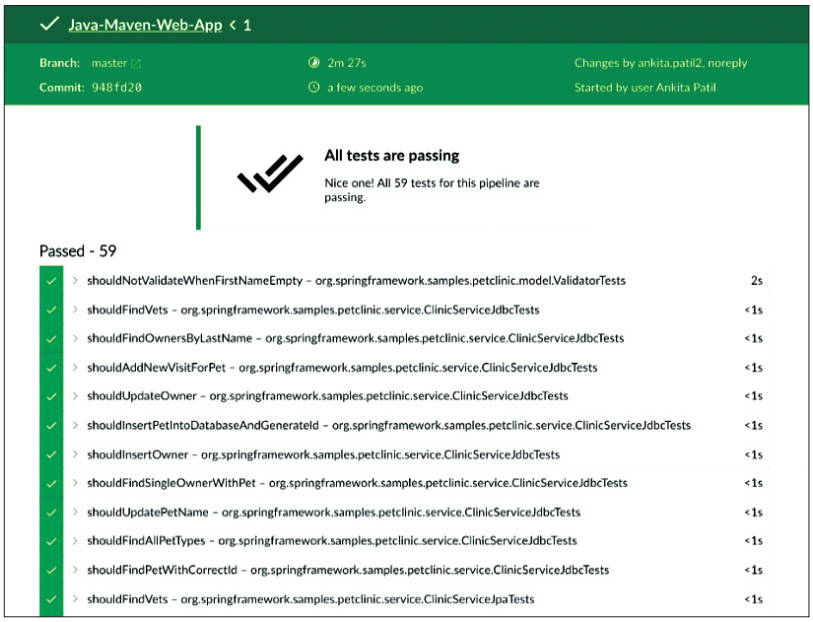

Figure 3: Unit test report on Jenkins dashboard

```
            <version>3.6.0.1398</version>
        </plugin>
    </plugins>
</build>
```

Now, to create a Jenkins pipeline, create a *Jenkinsfile* in the source code repository. Add the script below to implement unit test cases and code coverage:

```
pipeline {
agent {
    node {
      label 'master'
    }
  }
  stages {
    stage('Continuous Integration') {
      steps {
        withMaven(jdk: 'JAVA_HOME', maven: 'MAVEN_HOME') {
          bat 'mvn clean'
          bat(script: 'mvn test cobertura:cobertura install',
label: 'Unit Testing and Code Coverage')
          cobertura(autoUpdateHealth: true,
autoUpdateStability: true, classCoverageTargets: 'target/site/
cobertura/', coberturaReportFile: 'target/site/cobertura/*.
xml', failUnstable: true, zoomCoverageChart: true)
        }
      }
    }
  }
}
```

The *Jenkinsfile* in the code above includes the steps given below in the pipeline:

1. Execute unit test result and Cobertura code coverage. Get the reports in a SonarQube acceptable format (SonarQube supports JUnit, Cobertura and Jacoco reports).
2. Publish the unit test and coverage reports to the Jenkins dashboard. Save and run the pipeline in Jenkins again. On successful execution of unit testing, it will publish the results shown in Figure 3.

The code coverage report looks like what is shown in Figure 4.

Implementing SonarQube analysis from Jenkins pipeline: First, create the *sonar-project.properties* file in the root of the repository. This file is used to define the analysis parameter, which we need to provide to the SonarQube scanner during analysis. The sample *sonar-project.properties* file for our Maven application is in the code below:

```
sonar.projectKey=java-sonar-runner-sample
sonar.projectName=Simple Java project analyzed with the
SonarQube Runner
sonar.projectVersion=1.0

# Comma-separated paths to directories with sources
(required)
sonar.sources=src/main
sonar.tests=src/test
sonar.java.binaries=target/classes
sonar.java.test.binaries=target/test-classes

#Unit Test And Code Coverage
sonar.junit.reportPaths=target/surefire-reports
sonar.java.cobertura.reportPath=target/site/cobertura/
coverage.xml

sonar.coverage.jacoco.xmlReportPaths=target/site/jacoco/
jacoco.xml

sonar.sourceEncoding=UTF-8
```

Now, referring to this property file we need to perform SonarQube analysis from Jenkins. So update the *Jenkinsfile*, and add in it the code given below for the 'Continuous Integration' stage to perform SonarQube analysis (the code below has the entire *Jenkinsfile* for unit testing and SonarQube analysis):